

QUIC Server-Side Migration: Research Tasks

June 20, 2026

Task 1: Docker Setup and State Transfer Methods

Dockerize the migration setup and compare 4 methods for transferring the migration state from the primary server to the preferred server.

- 1a.** Convert network namespaces to Docker containers. Primary, preferred, and Redis each run in separate containers on a shared Docker network.
- 1b.** Shared volume (/tmp): primary writes the state to a shared Docker volume, preferred reads it. Baseline with no network transfer.
- 1c.** TCP push: primary opens a TCP connection to preferred and sends the state directly.
- 1d.** Redis: primary writes the state to a Redis instance, preferred retrieves it on demand.
- 1e.** QUIC-based transfer: primary sends the state inside a QUIC connection to preferred. The transfer is encrypted and looks the same as regular QUIC traffic to an observer.
- 1f.** Measure transfer latency, migration success rate, time-to-first-byte after migration, and whether the transfer is visible in packet captures (tcpdump/Wireshark).

Task 2: Pass-Through Handshake (QUIC Proxy)

Instead of exporting crypto state after the handshake, can the primary forward TLS handshake messages to the preferred server in real time so that preferred derives its own keys?

- 2a.** Study how NSS (Nemo's TLS library) handles key derivation internally: how it goes from the ECDHE shared secret to handshake keys to traffic keys, and at which points in the pipeline state can be extracted or injected.
- 2b.** Check whether preferred can derive TLS keys by observing the forwarded handshake packets without any additional information from primary.
- 2c.** Analyze the ECDHE problem: preferred does not have primary's ephemeral private key, so it cannot compute the shared secret from the forwarded public keys alone.
- 2d.** Using the NSS analysis from 2a, determine the minimum state primary needs to send (private key, shared secret, an intermediate derivation output) and whether this actually reduces what is exported or just changes when it is sent.
- 2e.** If a working approach exists, benchmark the latency against post-handshake export (Task 1).

Task 3: Applications of QUIC Migration

What is server-side migration actually useful for? Research each area below and identify open questions.

- 3a.** Identify the practical use cases: load balancing, failover, zero-downtime maintenance, geographic optimization. For each, determine whether the primary and preferred servers serve the same content (horizontal replicas) or different content (vertical separation).
- 3b.** Examine in which scenarios QUIC server migration has a clear advantage over existing load balancing approaches (reverse proxy, DNS-based, L4 load balancers). What does migration offer that these cannot?

- 3c. Our current migration exports only QUIC transport state (TLS secrets, connection IDs, packet numbers). Determine for which application types this is sufficient (e.g. static web, stateless APIs) and for which it is not (e.g. streaming, database-backed). For the cases where it is not, identify what additional state would need to be transferred.
- 3d. Explore whether stream-level load balancing is feasible. QUIC streams share encryption keys, packet numbers, congestion control, and ACK state at the connection level, and `preferred_address` migrates the entire connection. Investigate whether these are fundamental blockers or if workarounds exist (e.g. a proxy coordinating streams, a protocol extension).
- 3e. Evaluate multi-tier migration: `preferred_address` is set once during the handshake, so a connection cannot migrate twice. Each tier can only manage its own migrations independently.

Task 4: Migration vs. Industry Approaches

No production deployment uses `preferred_address` for server migration. Industry (Google, Cloudflare, Facebook) uses connection ID-based routing instead (QUIC-LB, RFC 9312). Why, and what does migration offer that CID routing cannot?

- 4a. Study how QUIC-LB works (RFC 9312): the load balancer reads the Connection ID from each packet (no decryption needed) and routes to the correct backend. No state transfer, no crypto export, load balancer stays in the path.
- 4b. Compare CID-based routing against `preferred_address` migration: what are the tradeoffs in latency, security, scalability, and architectural complexity? What can migration do that CID routing cannot (e.g. the front-end fully exiting the data path)?
- 4c. Investigate what modifications a QUIC library needs to support cross-machine migration. In Neco, this required changes to crypto state export (via NSS), connection ID management, and state serialization. Would other QUIC implementations (Cloudflare quiche, msquic, nginx's QUIC) need similar changes?
- 4d. Evaluate what a production server (e.g. nginx, which is open source) would need to support `preferred_address` migration. Identify the architectural barriers: nginx uses OpenSSL (not NSS), stays in the data path as a proxy, and does not expose TLS key export APIs.

Task 5: Security Risks of Server-Side Migration

`preferred_address` is in RFC 9000 but almost nobody implements it. What security risks does it introduce, and in which deployments are those risks acceptable?

- 5a. Document the attacks migration enables: client redirection to an attacker-controlled server, data exfiltration through the preferred server, interception of the state transfer containing TLS secrets.
- 5b. Identify the verification gap: `PATH_CHALLENGE` confirms reachability but not identity. The client has no way to verify that the preferred server is operated by the same entity as the primary.
- 5c. Analyze which deployments have acceptable security risk (e.g. same-subnet replicas in the same trust domain) and which do not (e.g. cross-network, cross-jurisdiction). Use latency and success rate data from Task 1 to support the analysis.